

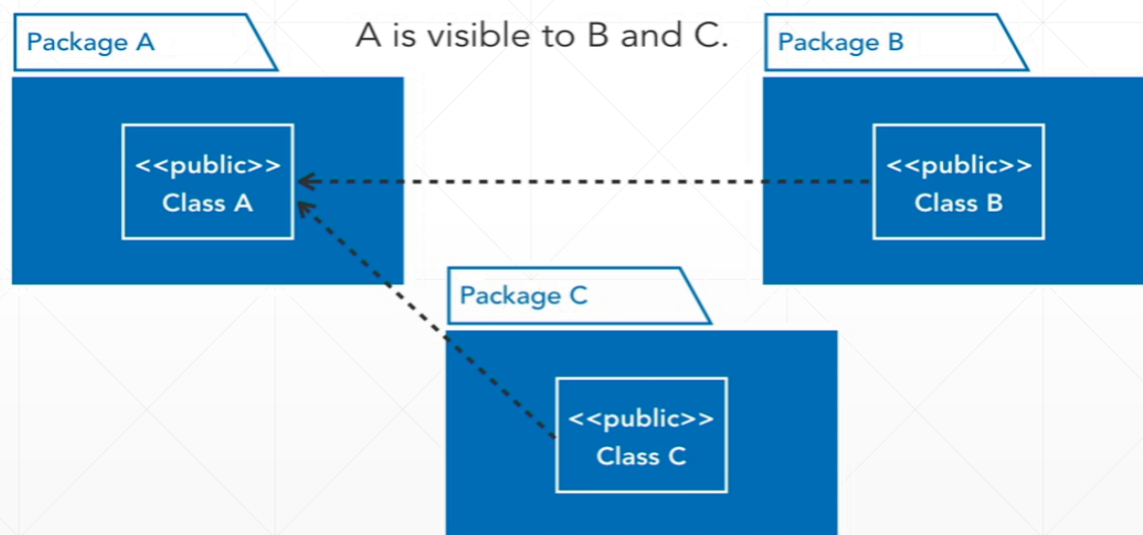


模块的传递性与聚合模块

北京理工大学计算机学院
金旭亮

类型的“可访问性”

在JDK9之前，一个声明为public的类，可以被外部代码随意地访问.....



模块的“强封装”特性



JDK 9之后，一个声明为public的类，并不一定就可以随意访问了，这要分情况进行讨论……



在JDK 9及以后的版本中，类声明为public，意味着它对同一模块中的其他包是可访问的，但只有当它所在的包也被本模块导出时，其他的模块才可以使用这个类。如果包没有被导出，外部是无法访问到它的，这个，被称为模块的“**强封装**”特性。

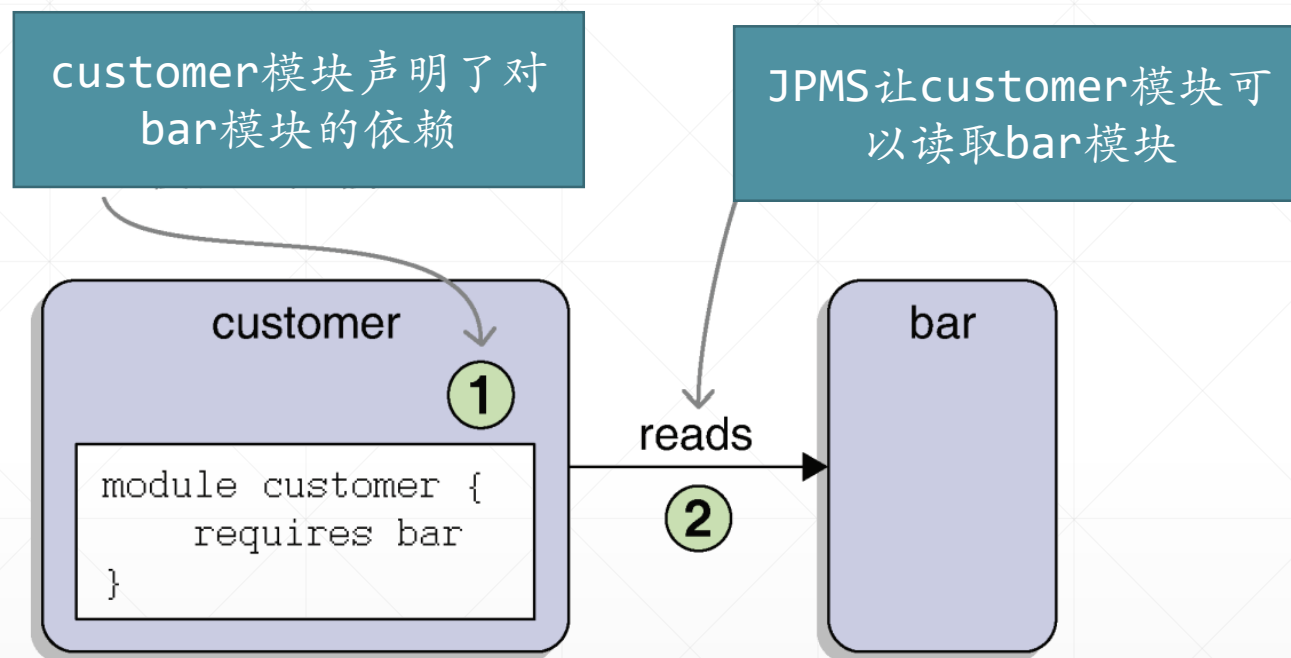


模块的“强封装”特性，对模块的“可读性”，有决定性的作用。



这就是强封装的意义所在。它迫使开发人员仔细设计包结构，将允许外部使用的类型与需要对外隐藏的内部实现细节代码分离开来。

模块可读性的概念



模块customer声明了需要bar，在运行时customer就可以读取bar模块，或反过来说，bar模块对customer模块来说是**可读**的。

确定可读性的条件



每当一个模块首次访问另外一个模块中的类型时，JPMS就会验证以下三个条件是否满足：

1

被访问的类型必须是公有的（`public`）

2

拥有该类型的模块必须已导出对应的包。

3

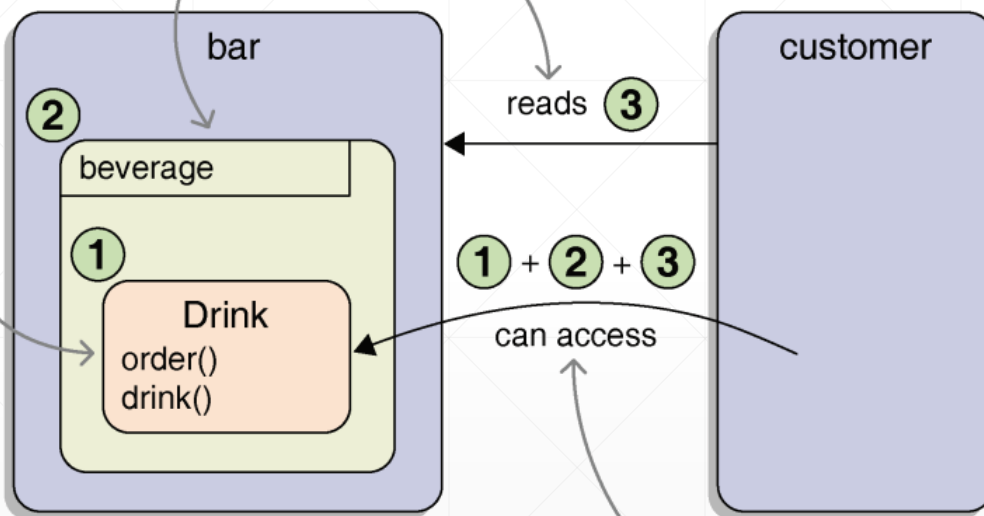
在Java程序启动时，会自动地创建一个模块依赖图，在此模块图中，访问模块必须连接到被访问的模块（即图中有一条可读边连接两个模块）。

确定类型的可访问性

Drink类是公有的...

...并且它所在的包已被模块导出...

...所以当其它模块读取bar模块时...



...Drink类是可以访问的...

如果某个类型不可访问，就表示无法与该类型进行交互：无法实例化它、无法访问它的字段、无法调用它的方法，也无法使用它的嵌套类。

理解“可读性”与“可访问性”



某模块可以读取其他模块，意味着它可以访问其他模块所导出包中的公有类型。模块之间的可读性关系是在`module-info.java`中使用`requires`子句设置的。



可读性关系表明了某模块可以读取其他哪些模块，但读取一个模块并不意味着就访问可以其导出包中的所有内容，这是可访问性所关注的地方。“可访问性”主要通过`public`、`private`和`protected`三个语言关键字来定义，它与可读性两者配合，共同确定某类型是否可以被外部模块的代码所访问。



可读性是“模块”级别的，可访问性是“类”和“字段/方法”级别的。

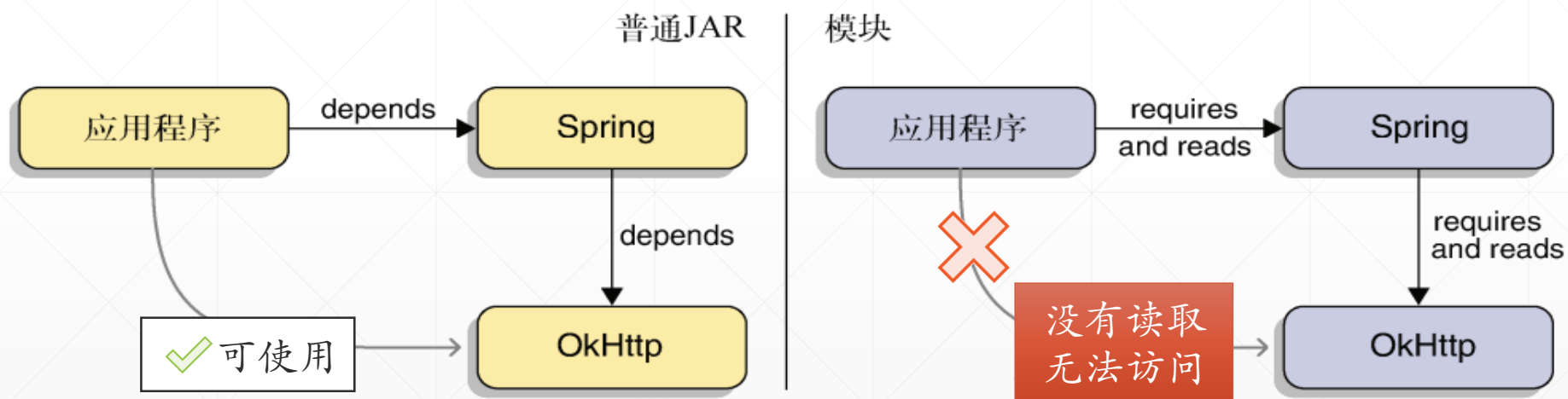
简言之.....



“模块可读”，并且“模块导出了特定的包”，再加上“类型是public”的，外部代码才可以创建这个类型的实例，这三个条件缺一不可。

依赖的传递性

早期的jar包，其依赖是可以传递的，因此“上游”jar中的代码，可以访问和读取“下游”jar包中的类型。



模块化之后，这种依赖的传递性是被阻止的，如果不作处理，“上游”模块中的代码，并不一定可以访问和读取到“下游”模块中的代码。

让模块依赖可传递



默认情况下，Java模块的依赖性是不可传递的。

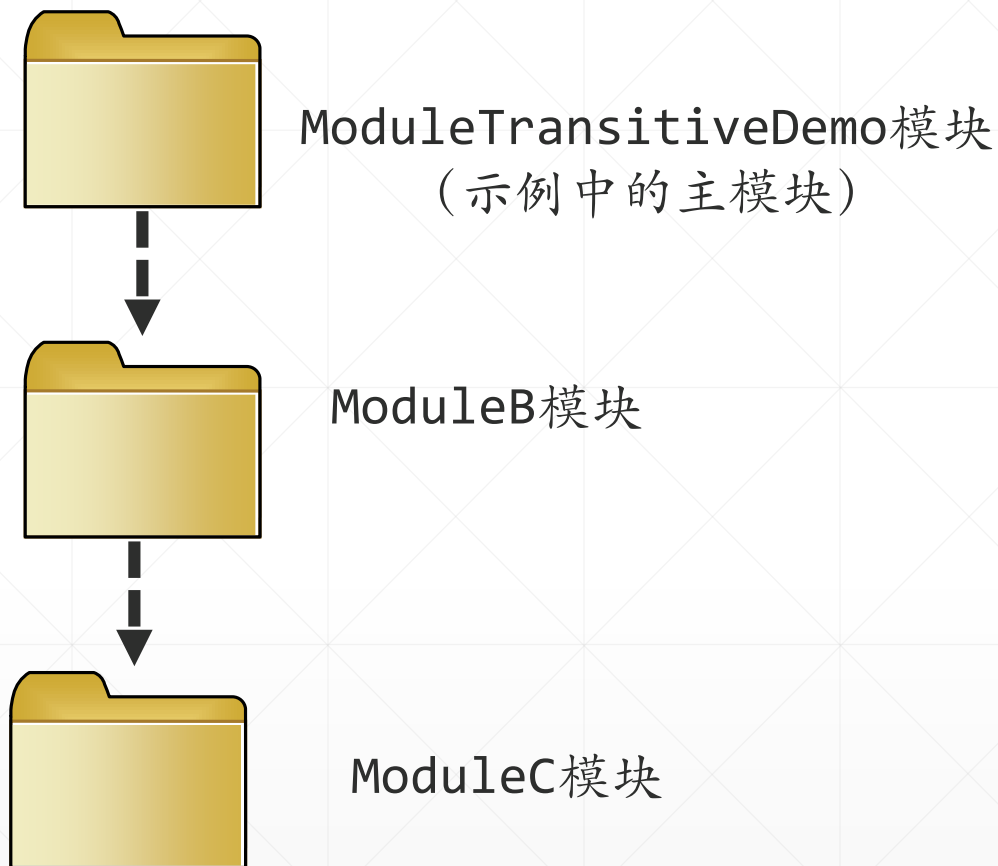
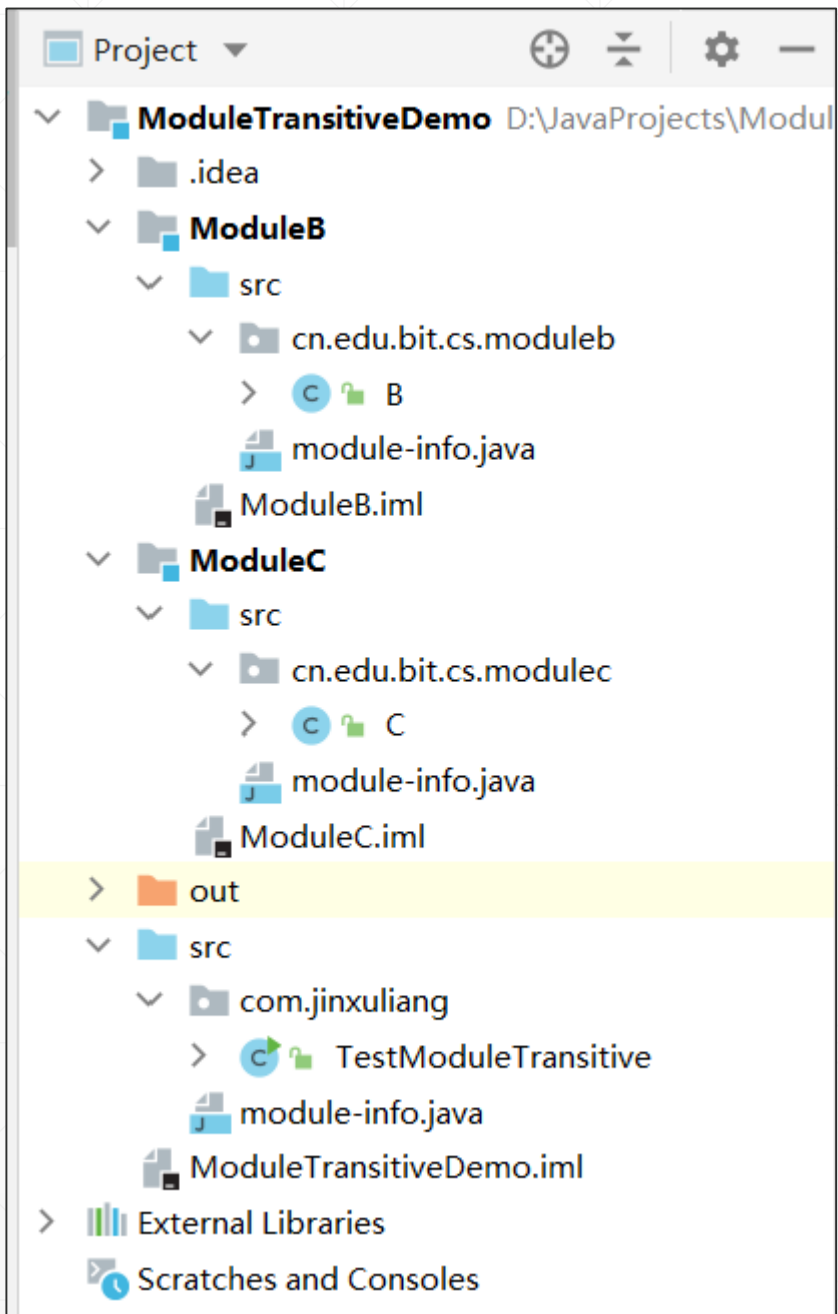


比如，模块A requires B，B requires C，并不意味着A就能读取C模块中的公有类型。



然而，如果B在其模块描述符中使用transitive声明requires，则A就能读取C中的类型了，这时，我们说A“**隐式依赖于**”C。

```
module B {  
    requires transitive C;  
}
```



示例程序中包容三个模块，彼此依赖。

ModuleC模块中的代码

```
C C.java x
1 package cn.edu.bit.cs.modulec;
2
3 import java.time.LocalDateTime;
4
5 public class C {
6     public LocalDateTime getCurrentTime(){
7         return LocalDateTime.now();
8     }
9 }
```

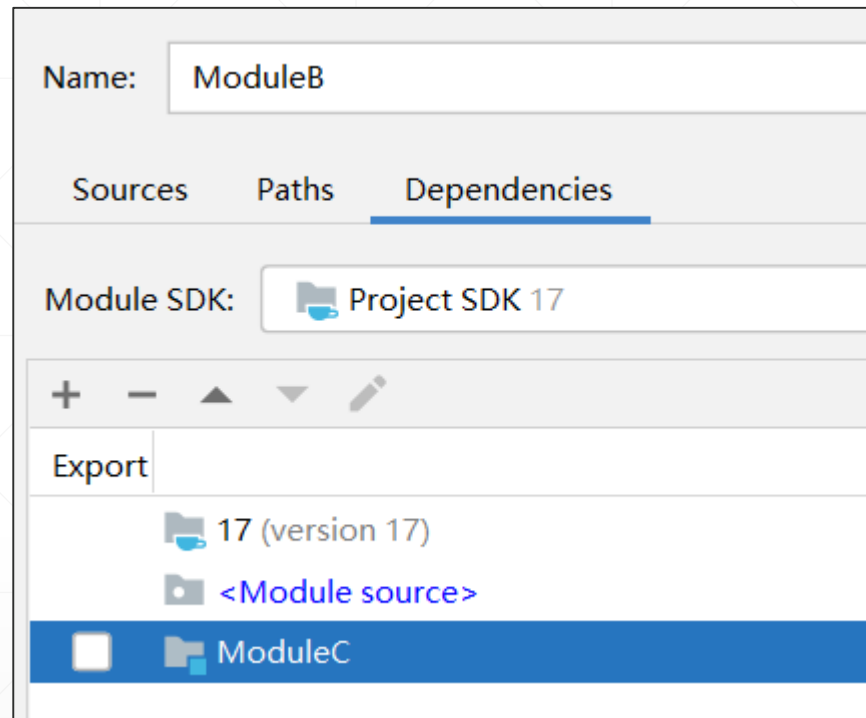
```
module-info.java x
1 module ModuleC {
2     exports cn.edu.bit.cs.modulec;
3 }
```

模块ModuleC中只有一个类C，它定义了一个公有方法，向外界返回当前的时间，注意它只导出了自己的包，并且不依赖于其它模块。

模块ModuleB中的代码

```
C B.java x
1 package cn.edu.bit.cs.moduleb;
2
3 import cn.edu.bit.cs.modulec.C;
4
5 public class B {
6     private C c=new C();
7
8     public C getC() {...}
11
12     public void setC(C c) {...}
15 }
```

```
module-info.java x
1 module ModuleB {
2     exports cn.edu.bit.cs.moduleb;
3     //声明对ModuleC模块的依赖是可以传递的
4     requires transitive ModuleC;
5 }
```

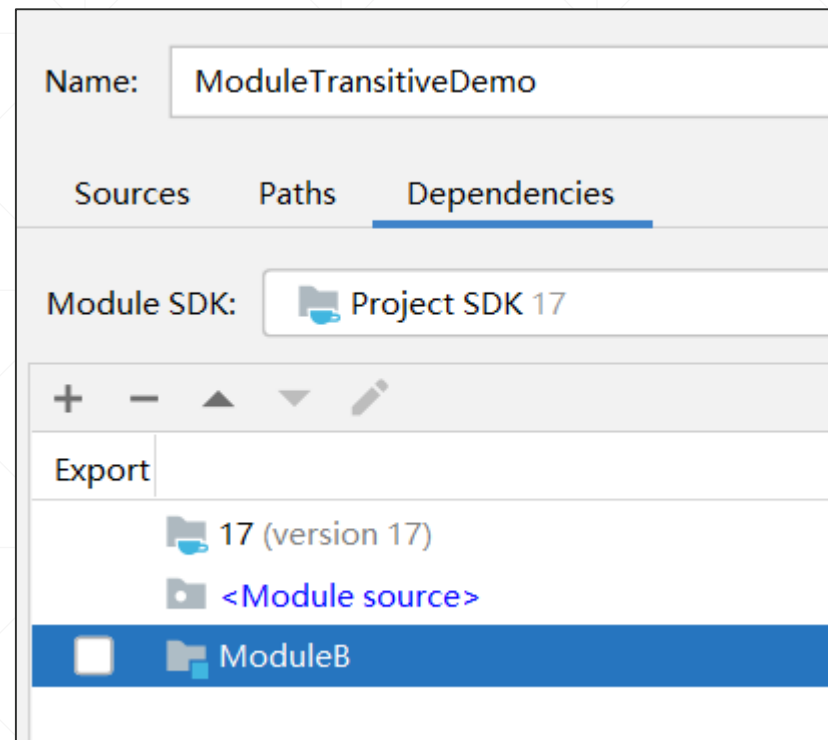


模块ModuleB依赖于模块ModuleC，其中的类B有一个属性，向外界返回一个C对象。注意模块ModuleB中的module-info.java里头的**transitive**关键字。

模块ModuleTransitiveDemo

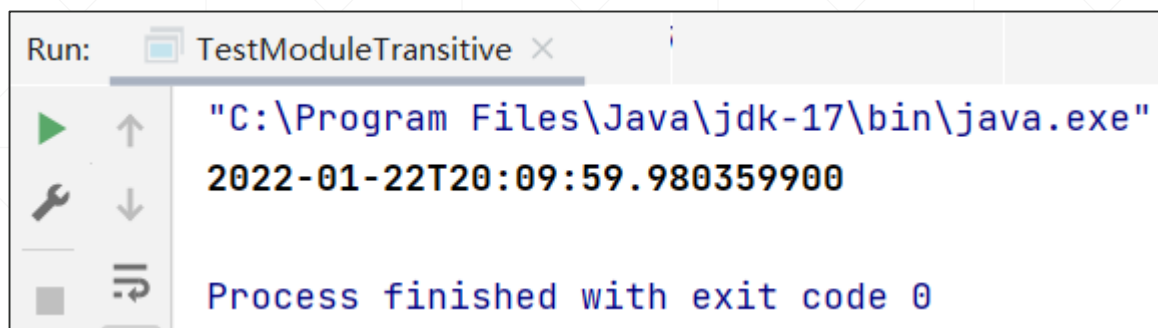
```
module-info.java x
1  module ModuleTransitiveDemo {
2      requires ModuleB;
3  }
```

```
TestModuleTransitive.java x
1  package com.jinxuliang;
2
3  import cn.edu.bit.cs.moduleb.B;
4
5  public class TestModuleTransitive {
6      public static void main(String[] args) {
7          var obj = new B();
8          System.out.println(obj.getC().getCurrentTime());
9      }
10 }
```



主模块ModuleTransitiveDemo只依赖于模块ModuleB，不需要声明它依赖于模块ModuleC，其原因在于ModuleB对ModuleC的依赖是可传递的，所以主模块中可以使用放在ModuleC中的类C。

运行截图



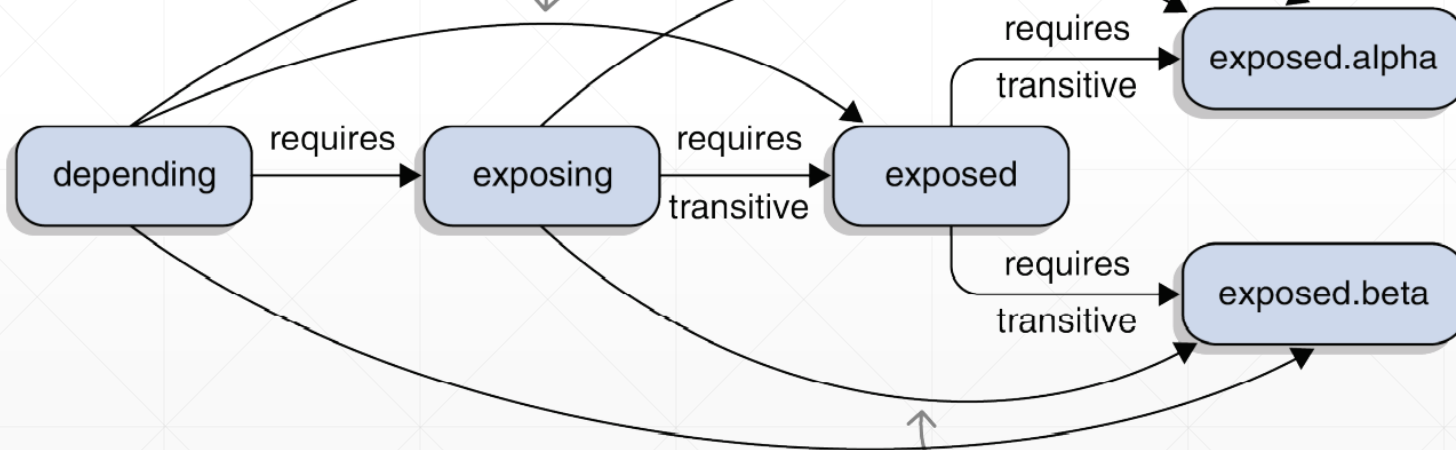
主模块通过读取B模块，获取了C模块中定义的对象，然后调用这个对象的相应方法，输出了当前的时间。

“依赖传递性链”

depending读取exposed.alpha是因为它能读取exposed, 后者隐式依赖于exposed.alpha

depending读取exposed是因为exposing隐式依赖于它

exposing读取exposed.alpha是因为它能读取exposed, 后者隐式依赖于exposed.alpha



基于依赖的传递性，使得位于链头的模块，可以读取它后面的所有模块。

depending读取exposed.beta是因为它能读取exposed, 后者隐式依赖于exposed.beta

exposing读取exposed.beta是因为它能读取exposed, 后者隐式依赖于exposed.beta

“模块传递性”应用实例：聚合器模块

有一种特殊的模块，大量地应用了transitive特性，比如JDK中的java.se模块，应用程序模块只要声明依赖于java.se，就能访问它所声明的各个模块。这种模块，称为“**聚合器 (Aggregator)**”模块。

```
module java.se {  
    requires transitive java.desktop;  
    requires transitive java.sql;  
    requires transitive java.xml;  
    requires transitive java.prefs;  
    //省略  
}
```



在应用程序模块中使用java.se是不明智的，这意味着在模块中复制了Java 9之前可以访问的rt.jar的所有类。在实际开发中，应该遵循“只添加用到的模块”，绝不“画蛇添足”的基本原则。

“聚合器模块”——java.se

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Users\JinXu> java --describe-module java.se
java.se@17
requires java.sql.rowset transitive
requires java.management.rmi transitive
requires java.rmi transitive
requires java.net.http transitive
requires java.compiler transitive
requires java.logging transitive
requires java.scripting transitive
requires java.xml.crypto transitive
requires java.base mandated
requires java.datatransfer transitive
requires java.naming transitive
requires java.sql transitive
requires java.xml transitive
requires java.security.sasl transitive
requires java.prefs transitive
requires java.management transitive
requires java.security.jgss transitive
requires java.transaction.xa transitive
requires java.desktop transitive
requires java.instrument transitive
PS C:\Users\JinXu> |
```

```
Windows PowerShell
PS C:\Users\JinXu> java --describe-module java.rmi
java.rmi@17
exports java.rmi
exports java.rmi.dgc
exports java.rmi.registry
exports java.rmi.server
exports javax.rmi.ssl
requires java.base mandated
requires java.logging
uses java.rmi.server.RMIClassLoaderSpi
qualified exports sun.rmi.server to jdk.management.agent
jdk.jconsole java.management.rmi
qualified exports sun.rmi.transport to jdk.management.agent
jdk.jconsole java.management.rmi
qualified exports sun.rmi.registry to jdk.management.agent
contains sun.rmi.log
contains sun.rmi.registry.resources
contains sun.rmi.runtime
contains sun.rmi.transport.tcp
PS C:\Users\JinXu> |
```

java.se模块requires java.rmi，并且是可传递的，这意味着依赖于java.se模块的项目，可以直接访问java.rmi模块中的类型。

平台模块中的隐式依赖

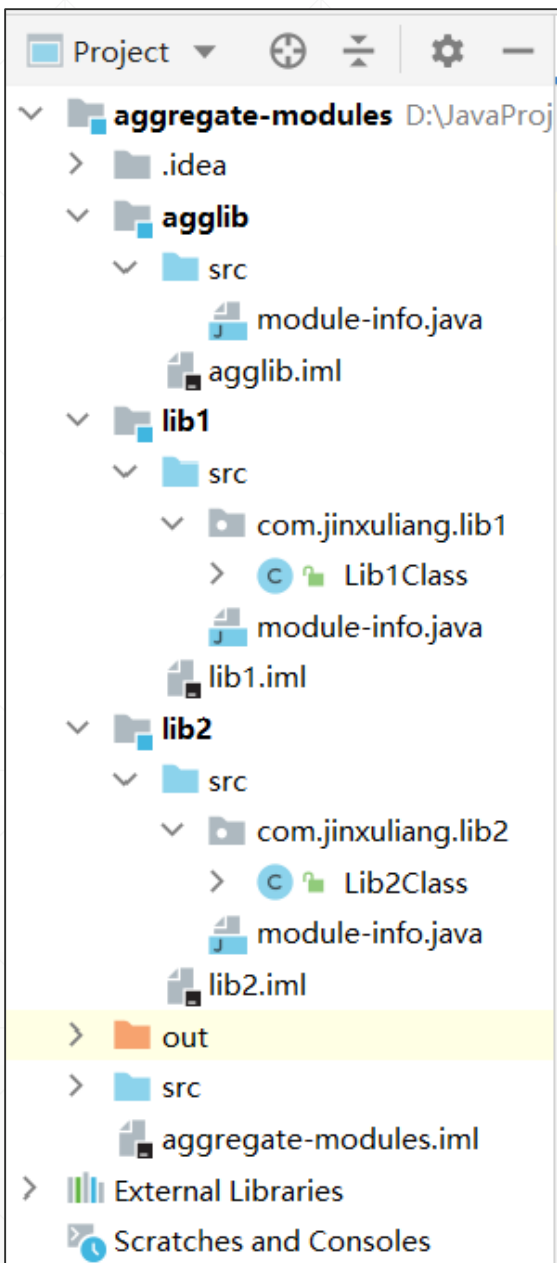
所有的模块，都依赖于java.base，这是隐式依赖，无需显式声明。

java.se和java.se.ee是两个重要的平台聚合器模块，可用于一次性定义多个隐式依赖。其中，java.se.ee传递性地依赖于java.se，这意味着，使用了JavaEE特性的Java应用，可以使用JavaSE的全部功能。

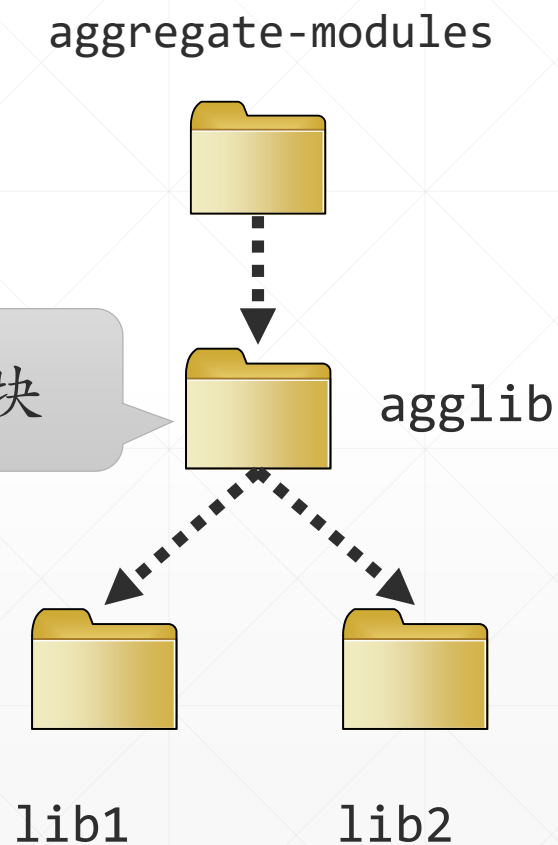
```
module java.se.ee {  
    requires transitive java.se;  
    requires transitive java.xml.ws;  
    requires transitive java.xml.bind;  
    //省略  
}
```

```
module java.se {  
    requires transitive java.desktop;  
    requires transitive java.sql;  
    requires transitive java.xml;  
    requires transitive java.prefs;  
    //省略  
}
```

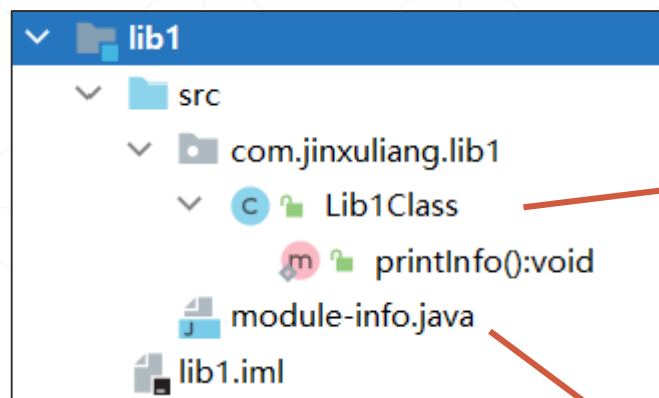
编写自己的聚合器模块



聚合器模块



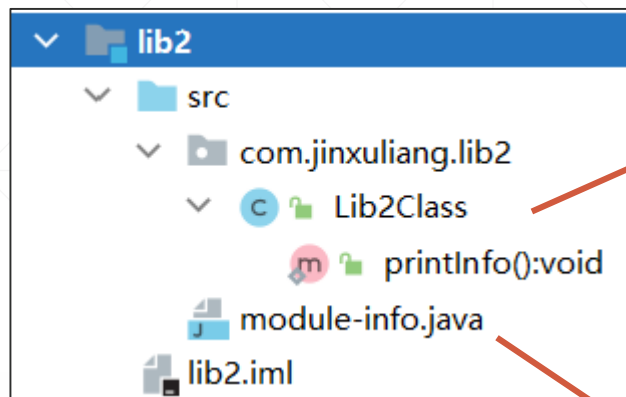
lib1模块



```
Lib1Class.java x
1 package com.jinxuliang.lib1;
2
3 public class Lib1Class {
4     public static void printInfo() {
5         System.out.println("Lib1Class.printInfo()");
6     }
7 }
```

```
module-info.java x
1 module lib1 {
2     exports com.jinxuliang.lib1;
3 }
```

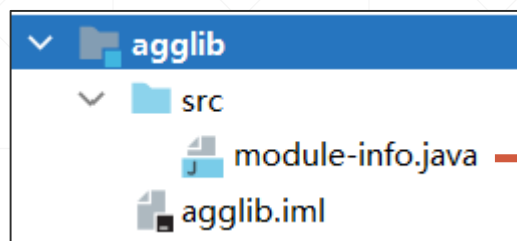
lib2模块



```
Lib2Class.java x
1 package com.jinxuliang.lib2;
2
3 public class Lib2Class {
4     public static void printInfo() {
5         System.out.println("Lib2Class.printInfo()");
6     }
7 }
```

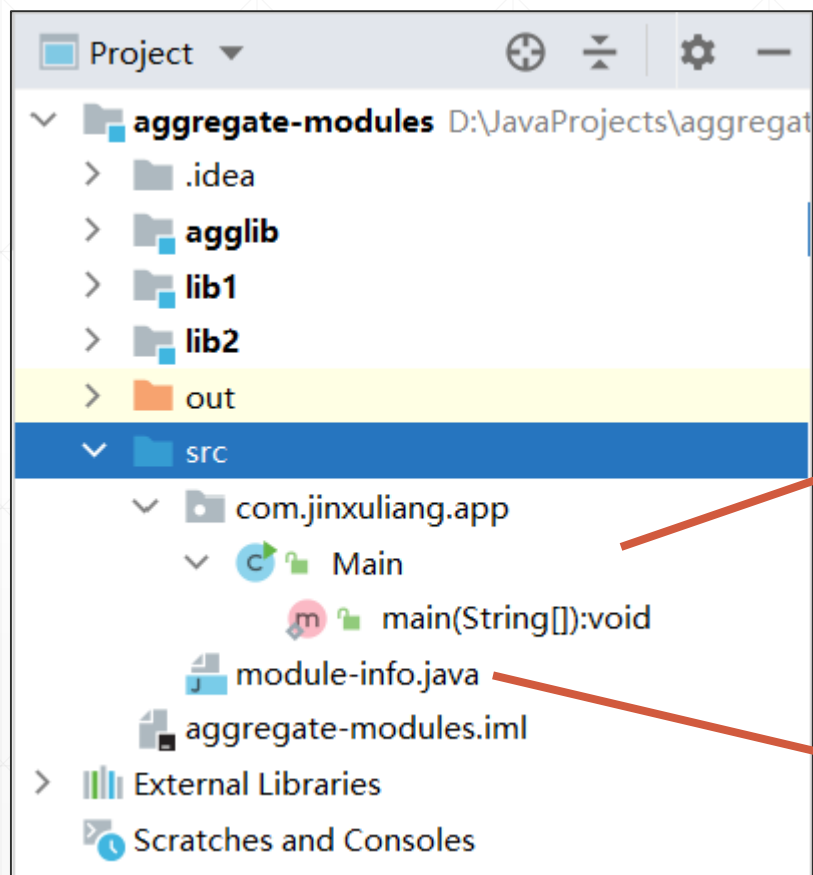
```
module-info.java x
1 module lib2 {
2     exports com.jinxuliang.lib2;
3 }
```

聚合器模块agglib



```
module-info.java x
1  module agglib {
2      requires transitive lib1;
3      requires transitive lib2;
4  }
```

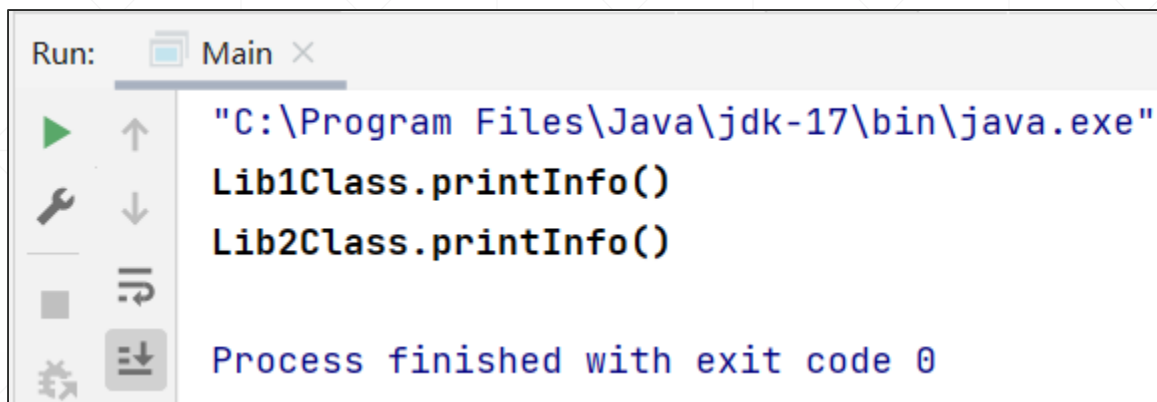
顶层模块



```
Main.java x
1  package com.jinxuliang.app;
2
3  import com.jinxuliang.lib1.Lib1Class;
4  import com.jinxuliang.lib2.Lib2Class;
5
6  public class Main {
7      public static void main(String[] args) {
8          //通过聚合模块使用Lib1和Lib2中的类
9          Lib1Class.printInfo();
10         Lib2Class.printInfo();
11     }
12 }
```

```
module-info.java x
1  module aggregate.modules {
2      requires agglib;
3  }
```

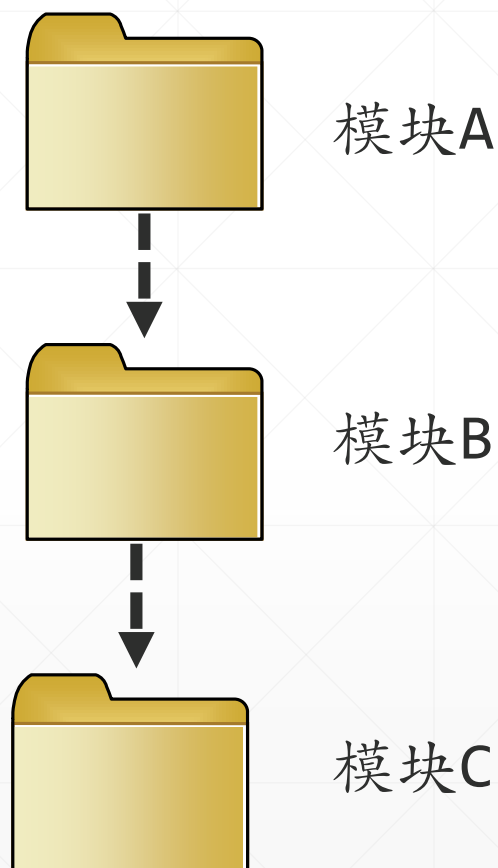

运行截图

A screenshot of an IDE's Run console window. The window has a title bar that says "Run: Main x". On the left side of the console, there is a vertical toolbar with icons for running (a green play button), stepping through code (a wrench and a downward arrow), and other debugging actions. The main area of the console displays the following text in blue:

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
Lib1Class.printInfo()  
Lib2Class.printInfo()  
  
Process finished with exit code 0
```

主模块通过agglib这个聚合器模块，成功地调用了它所聚合的lib1和lib2两个模块中的代码。

注意明确使用依赖传递性的场景



场景一：A依赖B，B中的某些方法返回了定义在C中的类型，因此，A必须能够读取C，在这种情况下，可以使用依赖传递性简化代码。

场景二：A依赖于很多模块，可以使用依赖传递性编写出“聚合器模块”以简化代码。

场景三：B模块很大，希望将其拆分为多个小的模块，这时可以使用依赖传递性，将B转换为一个聚合器模块，这样，A的编译和运行，都不会受到任何的影响。

如果不是上述这几种场景，慎用此特性。